



Universidad Abierta Interamericana

Ingeniería en Sistemas · Trabajo de Diploma

TRABAJO DE DIPLOMA

Patrones de diseño

Builder · **Bridge** · **Chain of Responsibility**

INTEGRANTES

Cores Juan Ignacio · De Mieri Facundo · Ricci
Agustín

DOCENTE

Leonel Jimenez Gamboa

COMISIÓN

T1-23-32 · Turno noche · 2026

Agenda

01 Builder

Creacional

3 ejemplos

02 Bridge

Estructural

2 ejemplos

03 Chain of Responsibility

De comportamiento

3 ejemplos

— Preguntas

01 / 03

Builder

Patrón creacional · GoF

Resuelve cómo se crean los objetos

El problema

```
new Producto(a);  
new Producto(a, b);  
new Producto(a, b, c);  
new Producto(a, b, c, d);  
new Producto(a, b, c, d, e, f, g);
```

Muchos parámetros, muchos opcionales, muchas sobrecargas

1 Constructor telescópico

Sobrecargas difíciles de leer, propensas a errores

2 Objetos a medio construir

Con setters sueltos queda inválido o incompleto

3 Lógica de armado duplicada

El mismo proceso, repetido por cada variante

Qué hace

Separa la construcción de un objeto complejo de su representación.

Paso a paso

El objeto se arma por partes, no con un único constructor

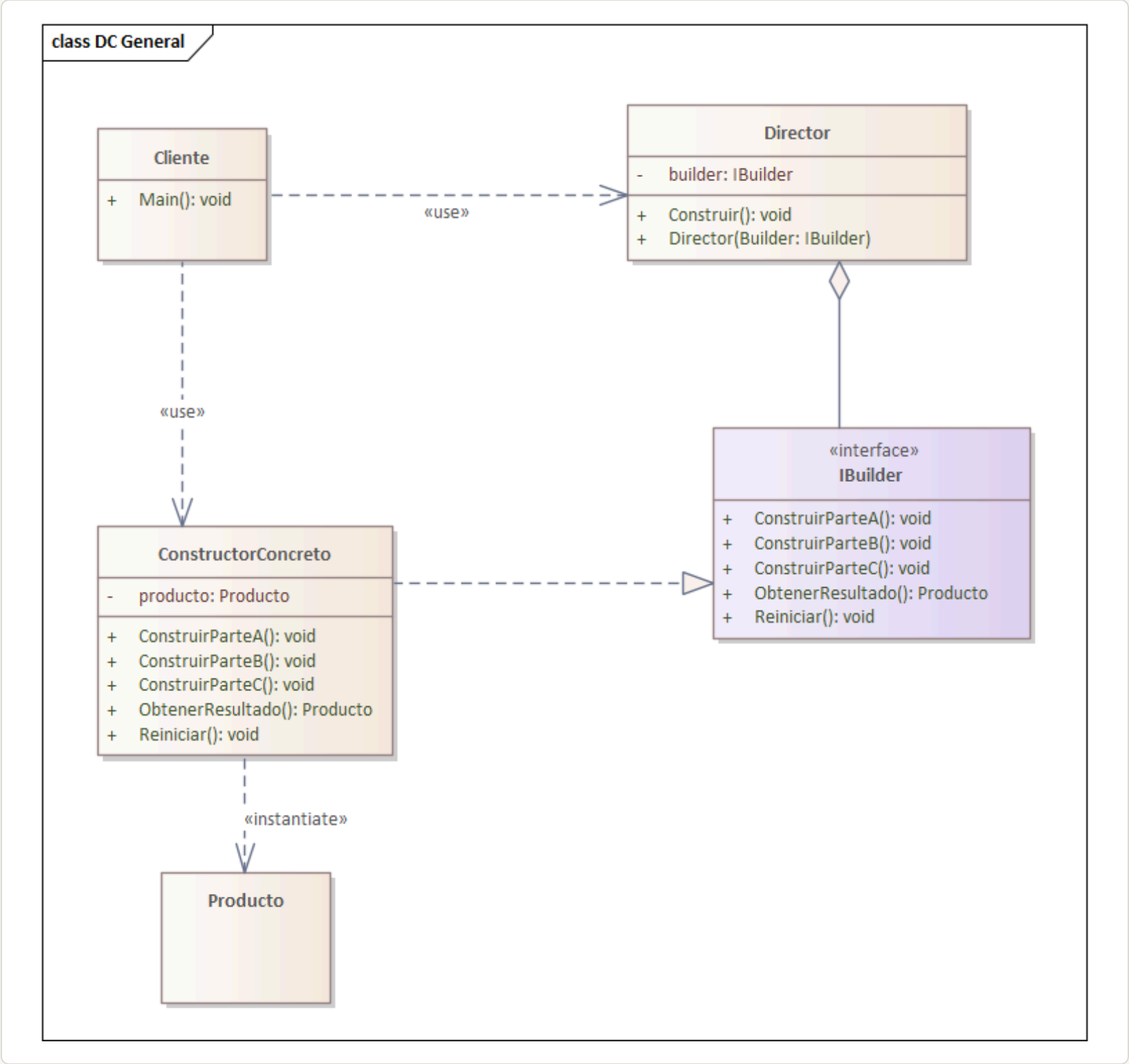
Un constructor por parte

Un objeto builder se encarga de cada parte

Completo y válido

El producto se entrega recién al terminar

Diagrama de clases general



Producto

Objeto complejo que se construye

Builder

Declara un método por cada parte

Constructor Concreto

Arma una representación y devuelve el producto

Director

Define el orden de los pasos

Cliente

Elige el constructor y obtiene el producto

Cuándo aplicarlo

Muchos atributos opcionales

Pasos en orden o con validaciones

Distintas representaciones del mismo proceso

Producto inmutable una vez creado

Ventajas

- + Construcción paso a paso
- + Mismo código, distintas representaciones
- + Responsabilidad Única
- + Cliente más legible

Desventajas

- Un builder por tipo de producto
- Complejidad innecesaria en objetos simples
- El builder duplica atributos del producto

Demo en vivo

Zapatilla personalizada

Builder · Ejemplo 1 → Visual Studio

Zapatilla personalizada

CONSIGNA

Modelo, color, suela y talla; el precio depende de lo elegido

Prearmada (Running o Urbana) o a medida por menú

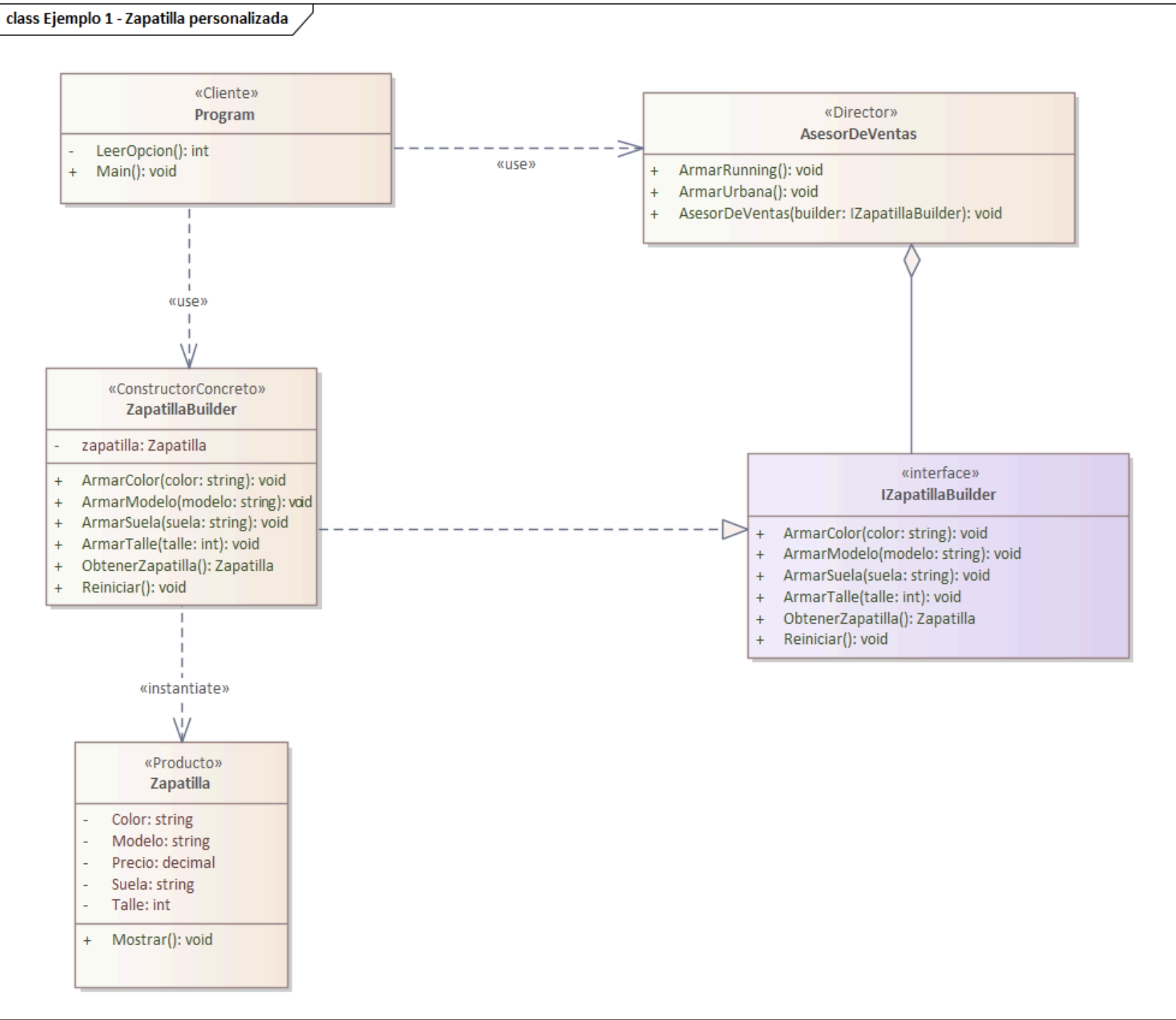
Muestra la ficha con su precio; talla entre 35 y 46

POR QUÉ BUILDER

Las prearmadas son recetas del Director (AsesorDeVentas)

A medida: el mismo builder, paso a paso

Un modelo nuevo no toca Zapatilla



Demo en vivo

Reportes en distintos formatos

Builder · Ejemplo 2 → Visual Studio

Reportes en distintos formatos

CONSIGNA

Reporte de ventas de un período, cargado por teclado

Salida en texto plano, HTML o CSV

Siempre: encabezado, detalle, totales y pie

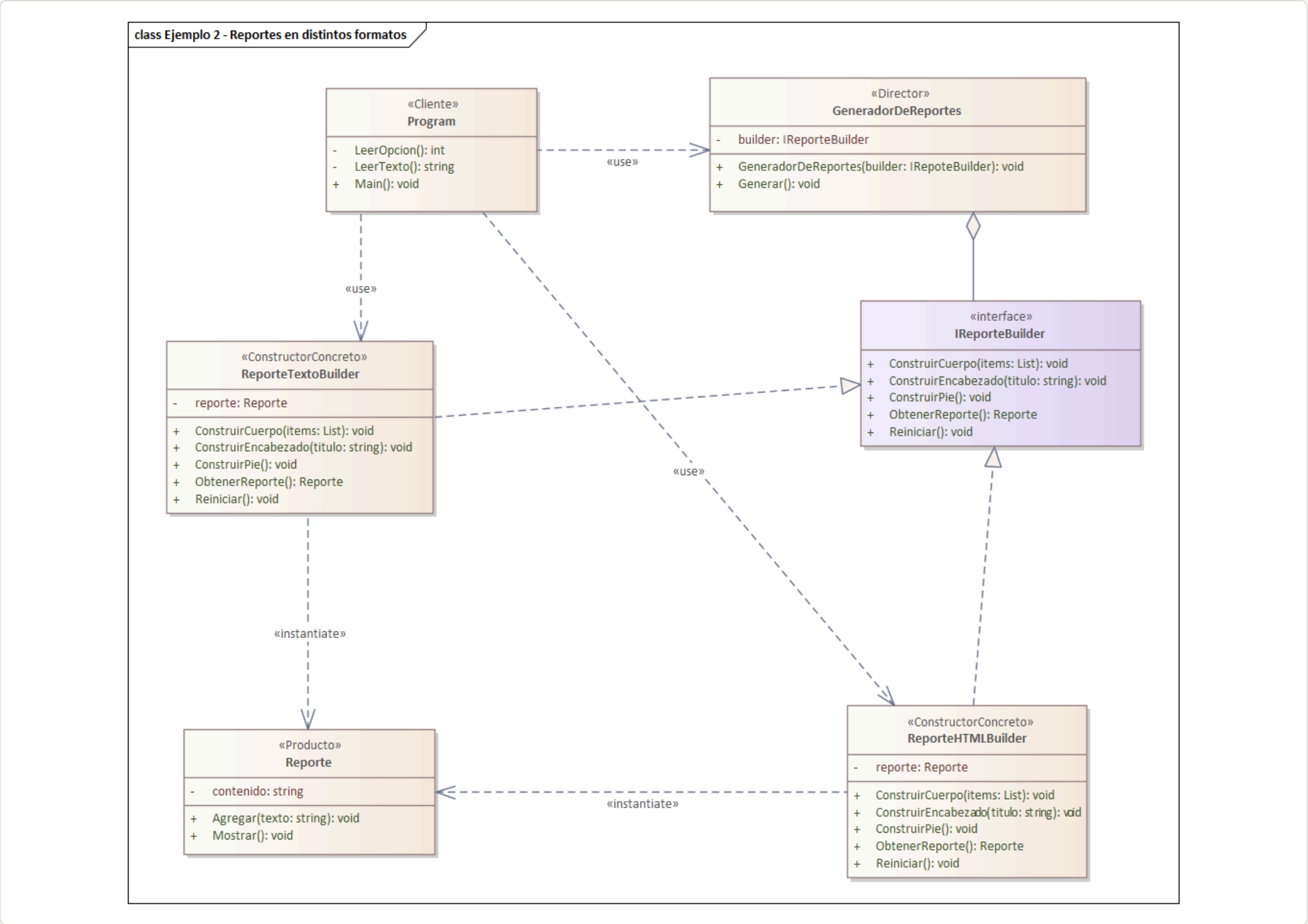
POR QUÉ BUILDER

GeneradorDeReportes repite los mismos pasos

Cada builder los representa en su formato

Formato nuevo = builder nuevo

(Abierto/Cerrado)



Setup de un auto F1

CONSIGNA

Siempre igual: alerones, suspensión, neumáticos

Mónaco: alta carga, suspensión y neumáticos blandos

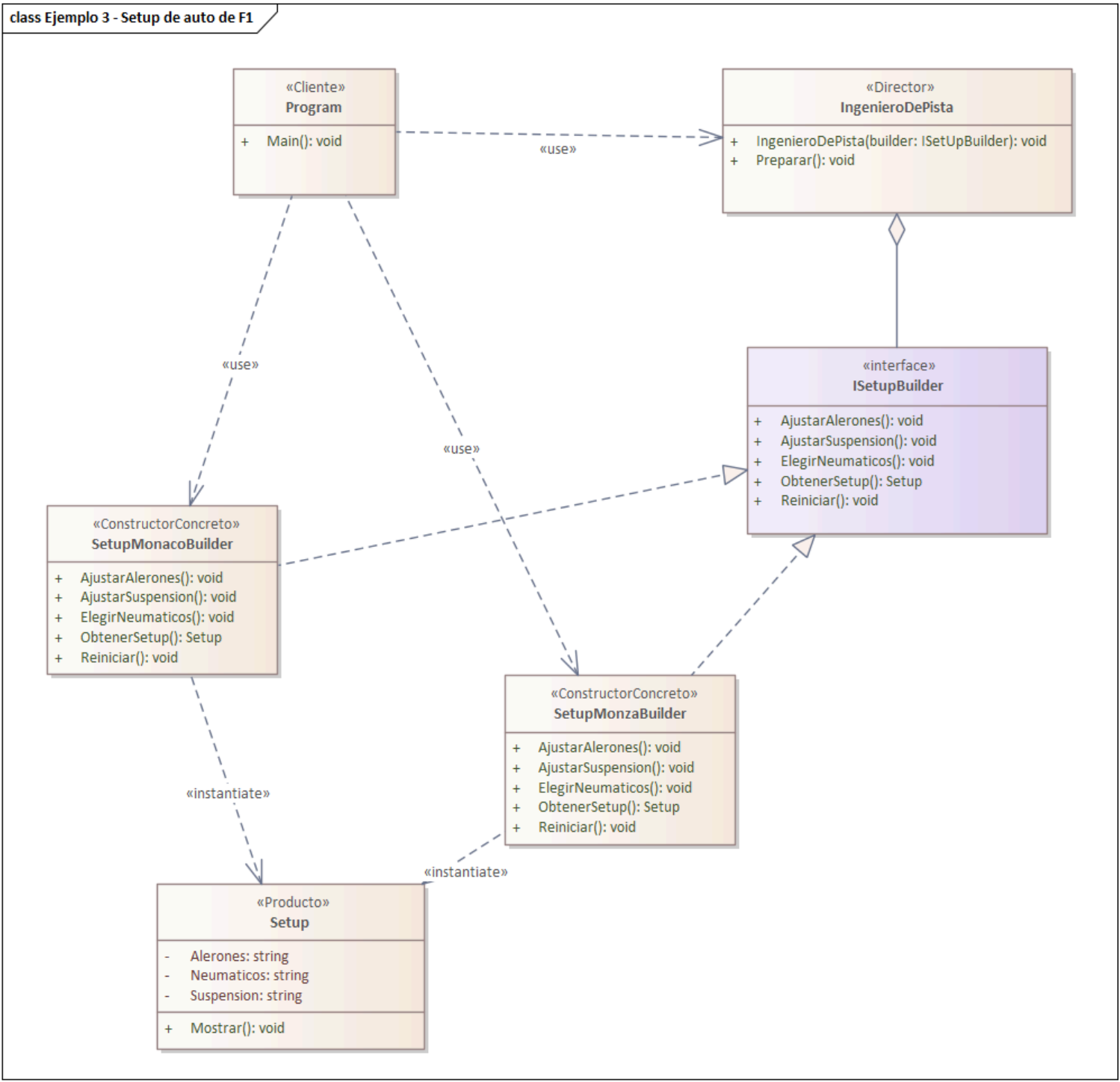
Monza: baja carga, suspensión y neumáticos duros

POR QUÉ BUILDER

IngenieroDePista ejecuta siempre los tres pasos

Cada builder resuelve los pasos para su circuito

Circuito nuevo (Spa) = un builder más



02 / 03

Bridge

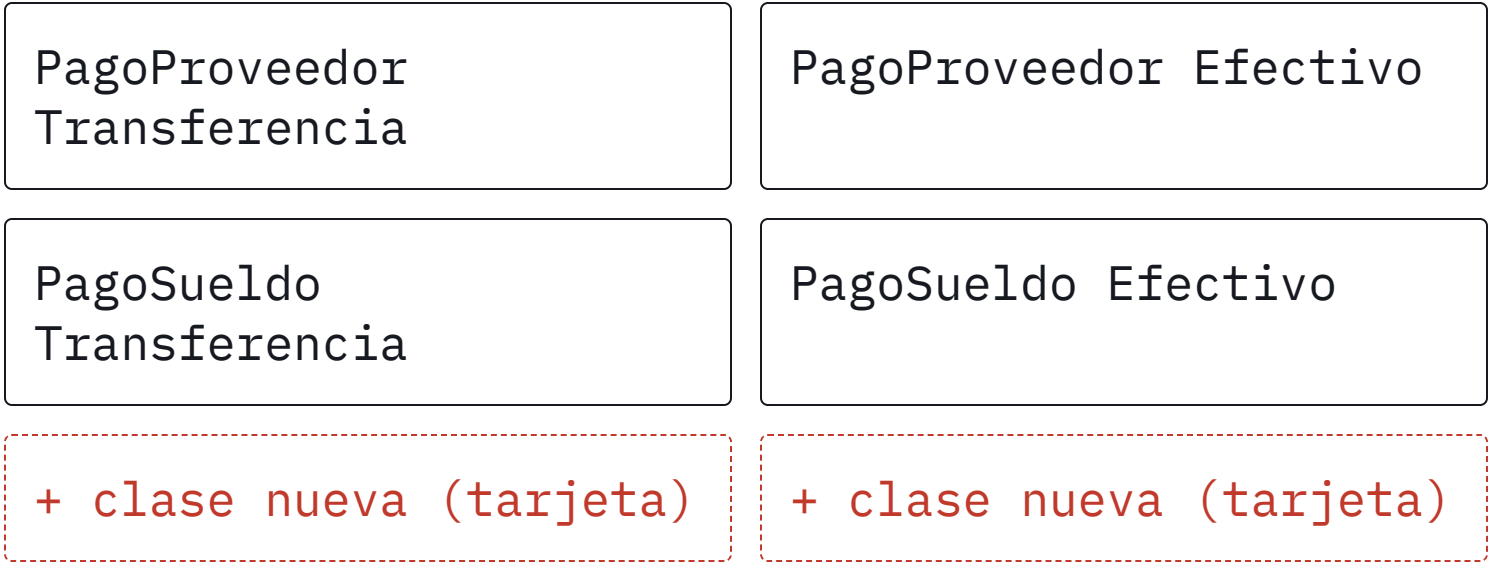
Patrón estructural · GoF

Resuelve cómo se combinan clases y objetos en estructuras más grandes

El problema

Con herencia

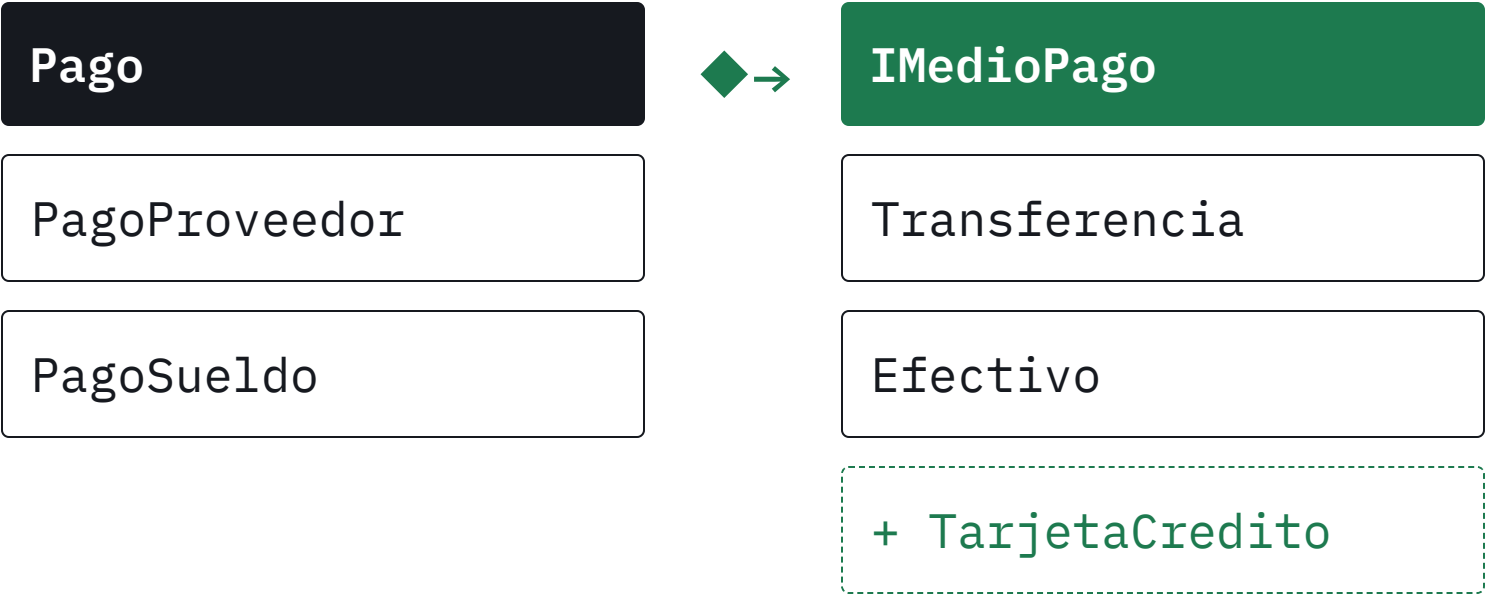
M × N



Un medio nuevo = **2 clases más**, una por tipo de pago

Con Bridge

M + N



Un medio nuevo = **1 sola clase**

Qué hace

**Separa una abstracción de su implementación,
para que ambas varíen por separado.**

Dos jerarquías

Unidas por composición, no por herencia

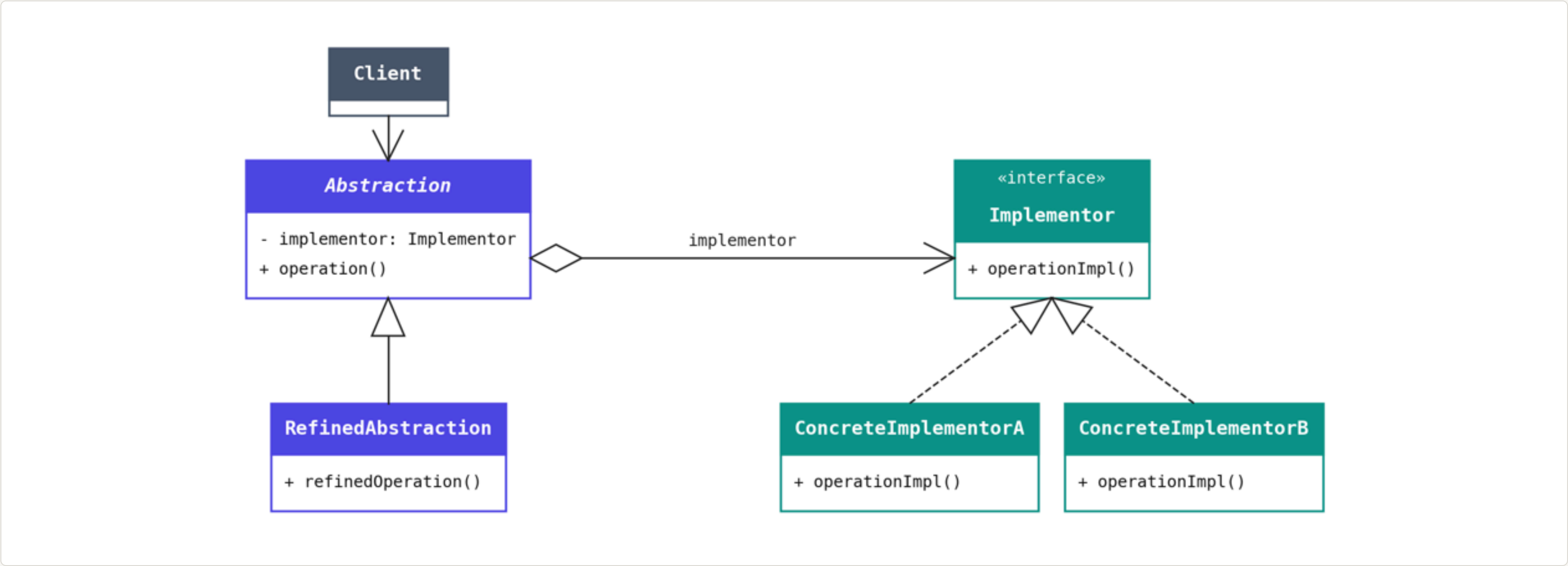
“Qué” y “cómo”

La abstracción delega el trabajo de bajo nivel en el implementador

El puente

Se recibe por constructor y puede cambiarse en ejecución

Diagrama de clases general



Abstracción

Alto nivel; delega en el implementador

Abstracción Refinada

Variantes de la lógica de alto nivel

Implementador

Interfaz de operaciones de bajo nivel

Implementador Concreto

Una variante específica

Cliente

Pasa el implementador por constructor

Cuándo aplicarlo

- Dos dimensiones de variación independientes
- Implementación intercambiable en ejecución
- Ambas jerarquías extensibles por separado
- Ocultar la implementación al cliente

Ventajas

- + Abstracción e implementación evolucionan aparte
- + $M + N$ clases en lugar de $M \times N$
- + Abierto/Cerrado y Responsabilidad Única
- + Implementación intercambiable en ejecución

Desventajas

- Más clases e interfaces
- Sobreingeniería con una sola dimensión
- Hay que identificar las dimensiones desde el diseño
- Indirección: el flujo es más difícil de seguir

Demo en vivo

Pagos de un ERP

Bridge · Ejemplo 1 → Visual Studio

Pagos de un ERP

CONSIGNA

Pagos a proveedores y de sueldos

Transferencia: 0,5 % (mín. \$5) · Efectivo: hasta \$5.000

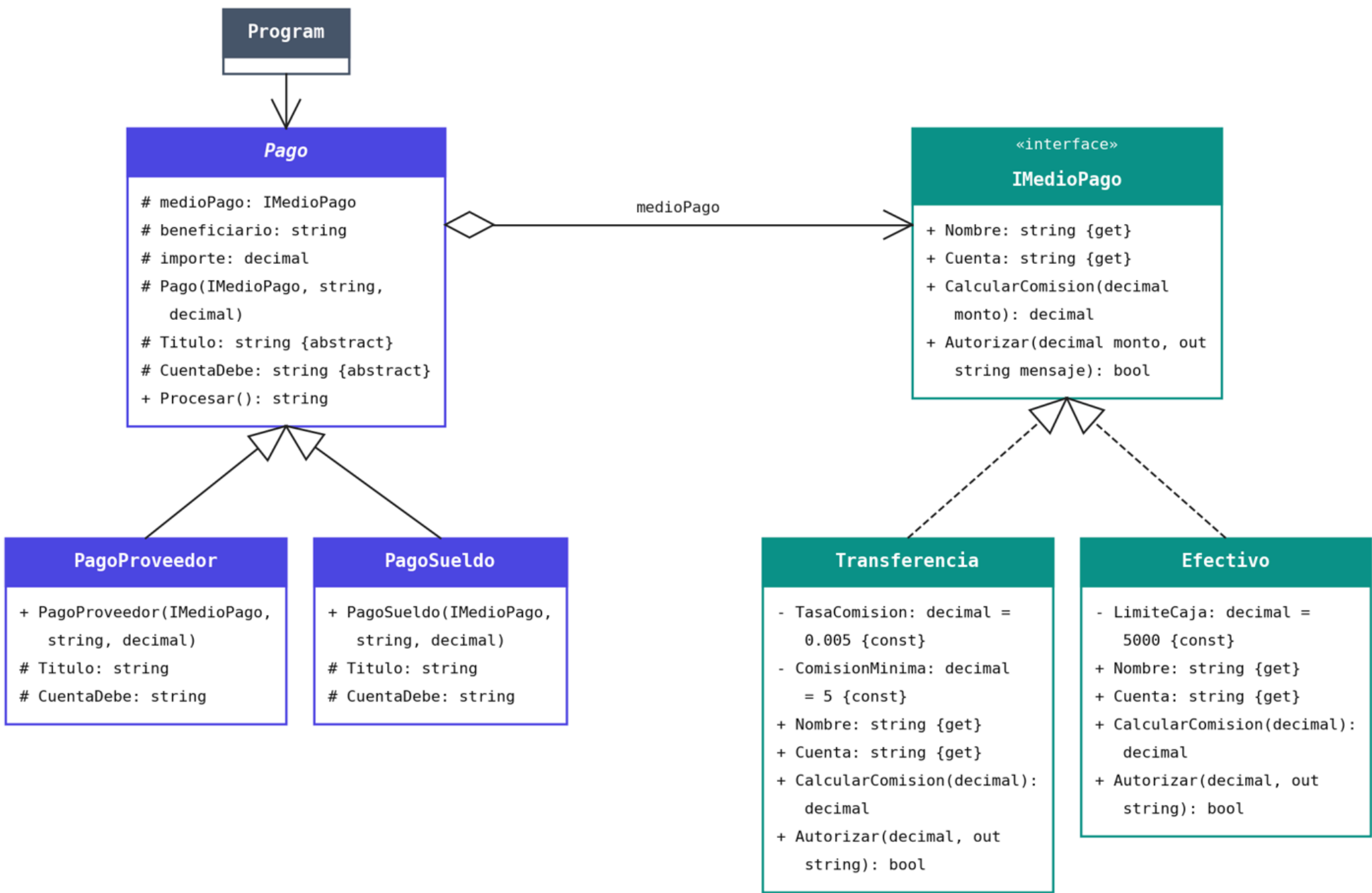
Si el medio autoriza, se genera el asiento

POR QUÉ BRIDGE

Dos dimensiones: tipo de pago × medio

Pago delega en IMedioPago comisión y autorización

TarjetaCredito = una sola clase nueva



Demo en vivo

Presupuestos y pedidos de venta

Bridge · Ejemplo 2 → Visual Studio

Presupuestos y pedidos de venta

CONSIGNA

Documentos: presupuesto y pedido de venta

Minorista sin descuento · Mayorista 15 % desde 20 u.

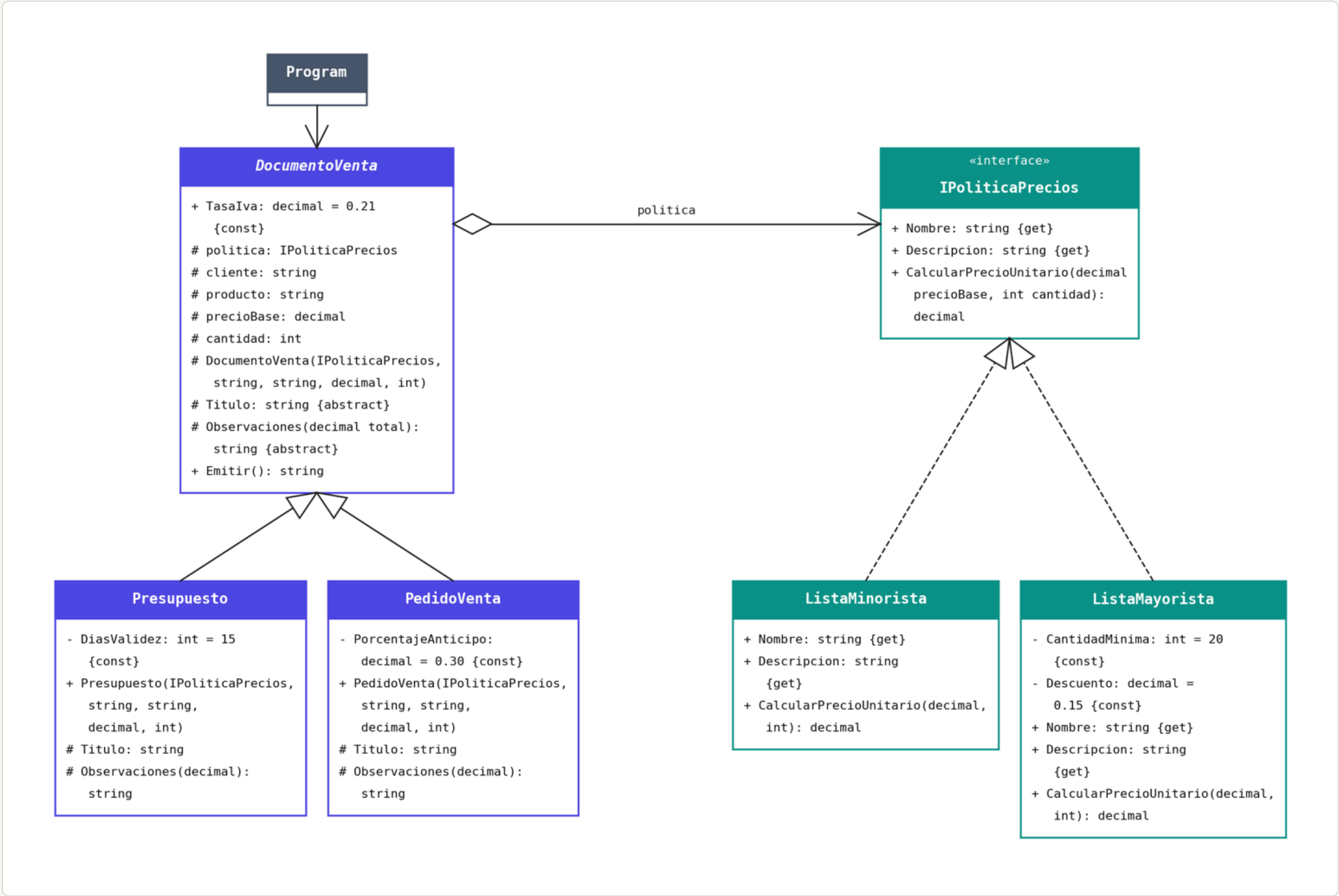
Bruto, descuento, neto, IVA 21 % y total

POR QUÉ BRIDGE

Dos dimensiones: documento × lista de precios

DocumentoVenta delega en IPoliticaPrecios

ListaCorporativa no toca documentos



03 / 03

Chain of Responsibility

Patrón de comportamiento · GoF

Resuelve cómo se reparten las responsabilidades y cómo se comunican los objetos

El problema

Sin el patrón

```
if (billetera alcanza)
    cobrar con billetera
else if (débito alcanza)
    cobrar con débito
else if (crédito alcanza)
    cobrar con crédito
else
    rechazar
```

El emisor conoce a todos los receptores y su orden

Con la cadena

Program → Pago



SaldoBilletera

↓ sucesor

TarjetaDebito

↓ sucesor

TarjetaCredito

El emisor solo conoce al primer eslabón

Qué hace

La solicitud pasa por una cadena de receptores hasta que alguno la procesa.

Emisor desacoplado

Envía al primer eslabón, no a un receptor concreto

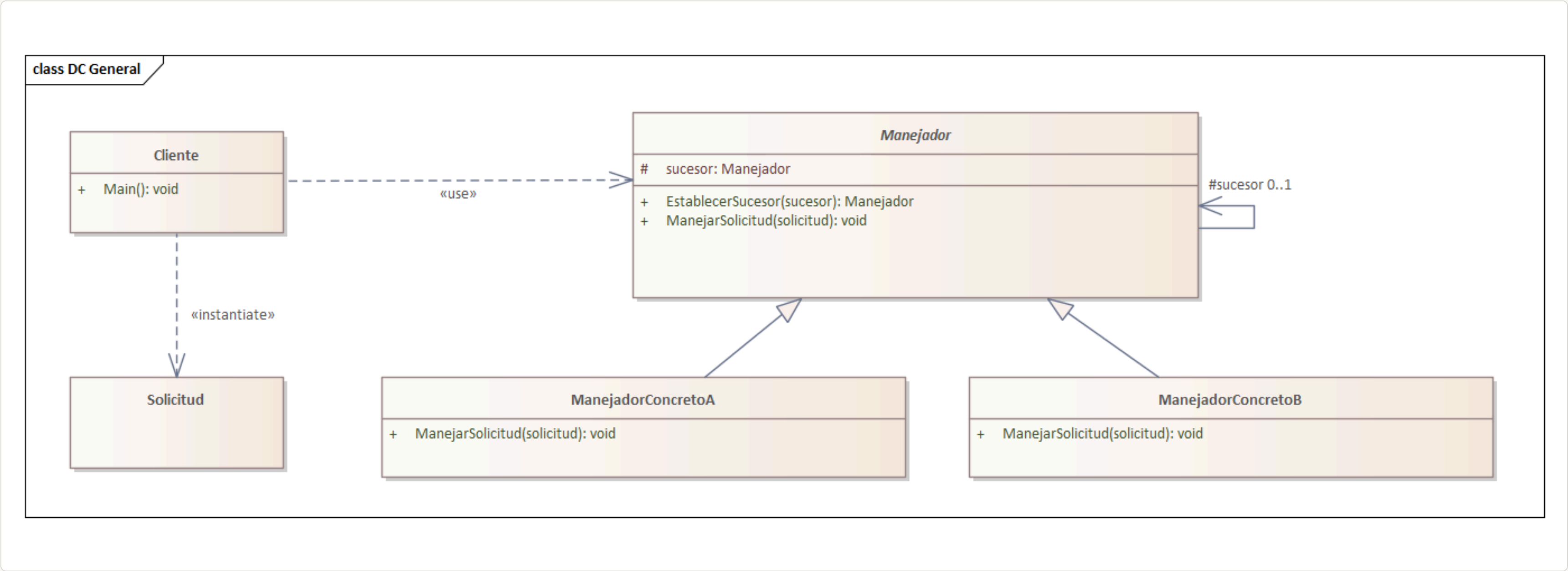
Solo el sucesor

Cada manejador no conoce la cadena completa

Procesa o pasa

Cada eslabón decide si atiende la solicitud

Diagrama de clases general



Solicitud

Datos del pedido que recorre la cadena

Manejador

Declara ManejarSolicitud() y guarda el sucesor

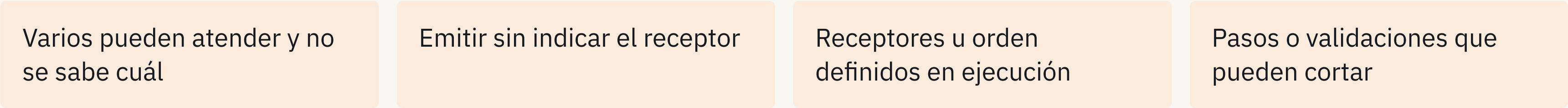
Manejador Concreto

Atiende lo suyo o lo pasa al sucesor

Cliente

Arma la cadena y envía al primero

Cuándo aplicarlo



Ventajas

- + Menor acoplamiento emisor–receptor
- + Abierto/Cerrado
- + Un criterio por manejador
- + Cadena configurable en ejecución

Desventajas

- Recepción no garantizada
- Más difícil de seguir y depurar
- Cadenas largas afectan el rendimiento

Demo en vivo

Pago con medios alternativos

Chain of Responsibility · Ejemplo 1 → Visual Studio

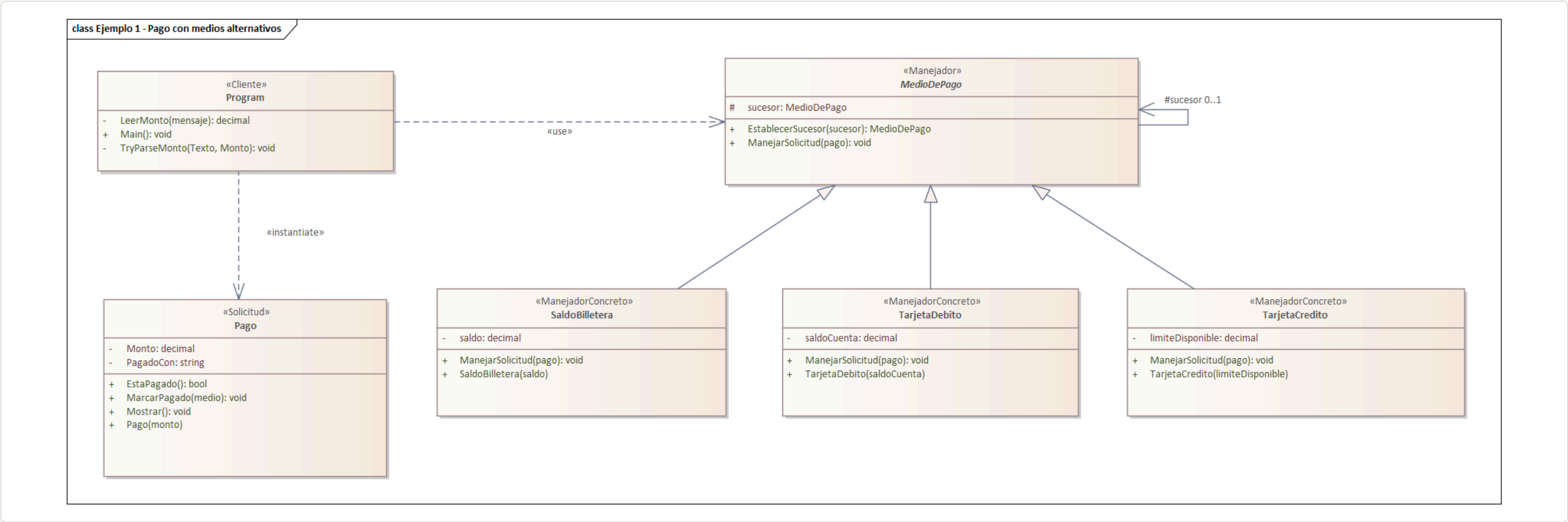
Pago con medios alternativos

CONSIGNA

Intenta billetera, débito y crédito, en ese orden
El primer medio que cubre el monto lo cobra
Si ninguno puede, el pago se rechaza

POR QUÉ CHAIN OF RESPONSIBILITY

Cada medio decide si puede cubrir el pago
Program solo conoce al primer medio
Medio nuevo = un eslabón más



Demo en vivo

Moderación de mensajes de un foro

Chain of Responsibility · Ejemplo 2 → Visual Studio

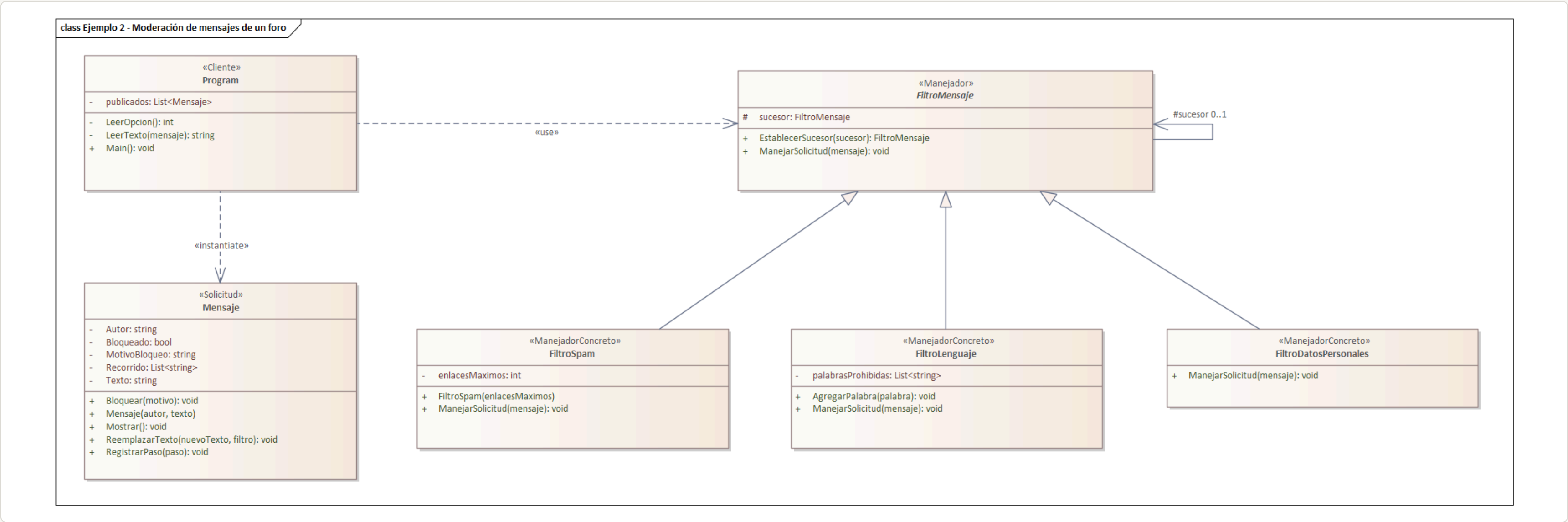
Moderación de mensajes de un foro

CONSIGNA

Filtros en orden: spam, lenguaje, datos personales
Spam bloquea; los otros modifican y dejan seguir
Si llega al final, se publica

POR QUÉ CHAIN OF RESPONSIBILITY

Cada eslabón procesa: corta o modifica y sigue
Control nuevo = una clase en la cadena
El orden importa: spam va primero



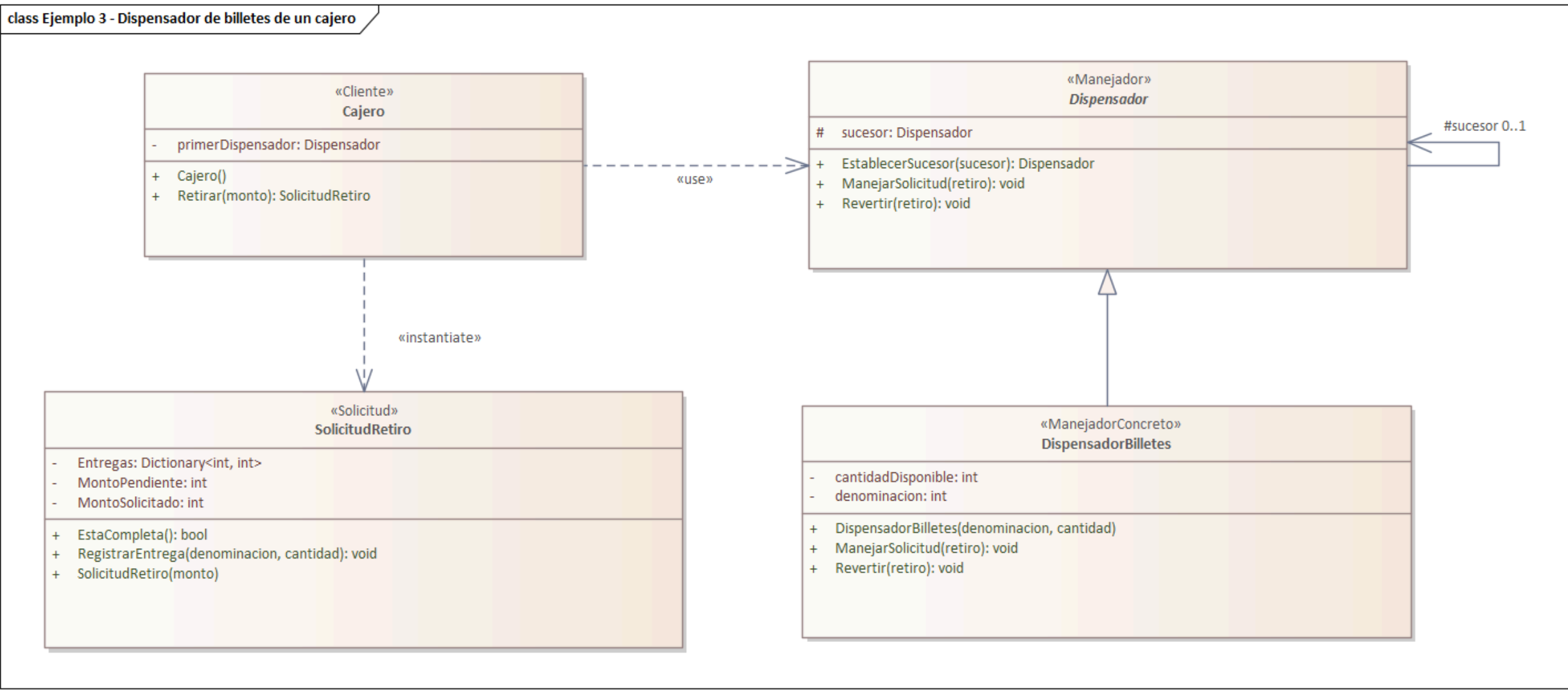
Dispensador de billetes de un cajero

CONSIGNA

- Billetes de \$20.000, \$10.000, \$2.000 y \$1.000
- Entrega primero los de mayor denominación
- Si no se forma el monto exacto, se cancela

POR QUÉ CHAIN OF RESPONSIBILITY

- Cada dispensador aporta su parte y pasa el resto
- Revertir() deshace sobre la misma cadena
- Cadena de objetos: misma clase, otra denominación





¿Preguntas?

REPOSITORIO

github.com/facudemieri/PatronesTD